MaterialApp dans Flutter

C'est quoi ?

`MaterialApp` est un **widget racine** qui enveloppe toute ton application Flutter. Tu le mets tout en haut de ton arbre de widgets, et il donne à ton app :

- Le **style Material Design** (couleurs, polices, animations, ripple effect au tap)
- La **navigation** entre écrans
- La **gestion des routes** (URLs internes)
- Les **thèmes** (clair/sombre)
- La **localisation** (langues)
- Le **titre** de l'app
- Un **debug banner** (le ruban rouge "DEBUG" en haut à droite)

Bref : sans lui, on a des widgets tout nus. Avec lui, on a une **vraie app** avec une identité visuelle cohérente.

Exemple minimal

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    MaterialApp(
      title: 'Mon App',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: Scaffold(
        appBar: AppBar(title: Text('Accueil')),
        body: Center(child: Text('Salut !')),
      ),
    ),
  );
}
```

`runApp()` prend **un seul widget** en paramètre → c'est presque toujours `MaterialApp` (ou `CupertinoApp` sur iOS, ou `WidgetsApp` si tu veux tout faire à la main).

Les paramètres qu'on va utiliser 90% du temps

Paramètre	Rôle
`home`	L'écran affiché au démarrage

Paramètre	Rôle
`initialRoute` / `routes`	Navigation par nom (`'/login'`, `'/home'`)
`onGenerateRoute`	Génération dynamique de routes (utile pour passer des arguments)
`theme`	Thème clair
`darkTheme`	Thème sombre
`themeMode`	`light`, `dark`, ou `system` (suit le téléphone)
`title`	Nom affiché dans le multitâche
`debugShowCheckedModeBanner`	`false` pour virer le ruban DEBUG
`navigatorKey`	Pour piloter la navigation depuis n'importe où
`locale` / `localizationsDelegates`	Gestion des langues
`builder`	Envelopper tout ton app (ex: MediaQuery, overlay)

Comment ça marche sous le capot

`MaterialApp` est en fait une **surcouche** au-dessus de `WidgetsApp`. Il ajoute :

1. Un `Theme` global (via `InheritedWidget`) → accessible partout avec `Theme.of(context)`
2. Les `Localizations` Material (boutons "OK", "Annuler", etc.)
3. Les transitions Material entre écrans (slide, fade…)
4. Les widgets comme `Scaffold`, `AppBar`, `FloatingActionButton` qui **dépendent** de lui pour fonctionner correctement

Si tu retires `MaterialApp`, `Scaffold` va râler : il ne trouve pas de `Material` ancestor.

Les pièges classiques

1. **Mettre `MaterialApp` plusieurs fois** → non, un seul à la racine. Pour un sous-arbre, utilise `Material` ou `Theme`.
2. **Oublier `home` ou `routes`** → écran noir, aucune erreur claire.
3. **`Navigator` avant `MaterialApp`** → erreur "No Navigator found". Le `Navigator` est **créé par** `MaterialApp`.
4. **Changer le thème à chaud** : utilise `themeMode` + un `ValueNotifier`, pas besoin de redémarrer.
5. **Confondre `MaterialApp` et `Scaffold`** : `MaterialApp` = config globale. `Scaffold` = structure d'**un** écran (AppBar + body + FAB + drawer).

Analogie simple

- `MaterialApp` = le **cadre + le règlement intérieur** de ton app (langue, couleurs, navigation)
- `Scaffold` = **une pièce** dans le bâtiment (salon, cuisine…)
- `AppBar`, `Body`, `FAB` = les **meubles** dans la pièce

Tu peux avoir plusieurs `Scaffold` (un par écran), mais un seul `MaterialApp`.

Quand NE PAS l'utiliser

- App **100% iOS-native** → `CupertinoApp`
- Jeu ou rendu custom total (Canvas, WebGL) → `WidgetsApp` seul ou `runApp` direct
- Test unitaire d'un widget → tu l'enveloppes manuellement avec `MaterialApp` **dans le test**, pas dans le code prod

Étudions les paramètres **un par un** pour qu’on l'effet de chacun.

Étape 0 — Le strict minimum

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('Mon App')),
        body: Center(child: Text('Bonjour !')),
      ),
    ),
  );
}
```

Ce que nous voyons :

- `runApp(...)` → démarre l'app
- `MaterialApp(...)` → la coquille qui donne le style Material
- `home:` → l'écran affiché au démarrage
- `Scaffold` → la structure d'un écran (barre du haut + corps)
- `AppBar` → la barre du haut
- `Center` → centre son enfant
- `Text` → affiche du texte

Retiens ça par cœur. Tout le reste, ce sont des options qu'on ajoute.

Étape 1 — Enlever le ruban rouge "DEBUG"

Par défaut, Flutter affiche un ruban rouge en haut à droite. Pour le virer :

```
MaterialApp(
  debugShowCheckedModeBanner: false,   //  ajoute cette ligne
  home: Scaffold(
    appBar: AppBar(title: Text('Mon App')),
    body: Center(child: Text('Bonjour !')),
  ),
)
```

Effet : plus de ruban rouge. C'est tout.

Étape 2 — Donner un titre à l'app

```
MaterialApp(
  title: 'Ma Super App',               //  ajoute cette ligne
  debugShowCheckedModeBanner: false,
  home: Scaffold(
    appBar: AppBar(title: Text('Mon App')),
    body: Center(child: Text('Bonjour !')),
  ),
)
```

Effet : le nom affiché dans le **multitâche** du téléphone (la liste des apps ouvertes). Pas visible dans l'app elle-même.

Étape 3 — Changer la couleur

```
MaterialApp(
  title: 'Ma Super App',
  debugShowCheckedModeBanner: false,
```

```
  theme: ThemeData(                    //  ajoute ce bloc
    primarySwatch: Colors.green,
  ),
  home: Scaffold(
    appBar: AppBar(title: Text('Mon App')),
    body: Center(child: Text('Bonjour !')),
  ),
)
```

Effet : l'AppBar devient verte, les boutons aussi. Essaie `Colors.red`, `Colors.purple`, `Colors.orange` pour voir.

Étape 4 — Ajouter un 2e écran (navigation simple)

Pour l'instant, **une seule page**. On va en ajouter une deuxième, et un bouton pour y aller.

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MonApp());
}

class MonApp extends StatelessWidget {
  const MonApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Ma Super App',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(primarySwatch: Colors.green),
      home: const PageAccueil(),
    );
  }
}

// --- Écran 1 ---
class PageAccueil extends StatelessWidget {
  const PageAccueil({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Accueil')),
      body: Center(
        child: ElevatedButton(
          child: const Text('Aller page 2'),
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(builder: (_) => const PageDeux()),
            );
          },
        ),
      ),
    );
  }
}
```

```
// --- Écran 2 ---
class PageDeux extends StatelessWidget {
  const PageDeux({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Page 2')),
      body: const Center(child: Text('Me voilà !')),
    );
  }
}
```

Ce qu'il faut comprendre ici :

- `Navigator.push(...)` → empile un nouvel écran par-dessus
- `MaterialPageRoute(...)` → la "route" qui contient l'écran à afficher
- Le bouton "retour" ← apparaît automatiquement dans l'AppBar

À ce stade, **tu n'as même pas besoin de `routes:`**. `Navigator.push` suffit pour 90% des cas débutant.

Étape 5 — Navigation par nom (plus propre)

Quand on a **beaucoup d'écrans**, écrire `MaterialPageRoute` partout devient lourd. On donne un **nom** à chaque écran :

```
MaterialApp(
  title: 'Ma Super App',
  debugShowCheckedModeBanner: false,
  theme: ThemeData(primarySwatch: Colors.green),

  initialRoute: '/',                    //  écran de départ
  routes: {                             //  la table des écrans
    '/': (context) => const PageAccueil(),
    '/deux': (context) => const PageDeux(),
  },
)
```

Et dans le bouton, on remplace :

```
// AVANT
Navigator.push(
  context,
  MaterialPageRoute(builder: (_) => const PageDeux()),
);

// APRÈS
Navigator.pushNamed(context, '/deux');
```

Effet : exactement le même, mais plus court à écrire. C'est **la seule** chose qu'on doit retenir sur `routes`.

Étape 6 — Mode sombre (light/dark)

C'est le seul cas où on a besoin de transformer `MonApp` en `StatefulWidget`, parce que le thème doit **changer** quand on clique.

```
class MonApp extends StatefulWidget {
  const MonApp({super.key});
  @override
  State<MonApp> createState() => _MonAppState();
}

class _MonAppState extends State<MonApp> {
  bool _sombre = false;                  //  état : clair ou sombre

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(primarySwatch: Colors.green),        // clair
      darkTheme: ThemeData(brightness: Brightness.dark),    // sombre
      themeMode: _sombre ? ThemeMode.dark : ThemeMode.light,
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Accueil'),
          actions: [
            IconButton(
              icon: const Icon(Icons.brightness_6),
              onPressed: () {
                setState(() {
                  _sombre = !_sombre;   //  on inverse
                });
              },
            ),
          ],
        ),
        body: const Center(child: Text('Bonjour !')),
      ),
    );
  }
}
```

Les 3 lignes clés :

- `theme:` → à quoi ça ressemble en clair
- `darkTheme:` → à quoi ça ressemble en sombre
- `themeMode:` → lequel choisir **maintenant**

`setState(...)` dit à Flutter : "j'ai changé une valeur, redessine l'écran".

Récap — Ce qu'on doit vraiment retenir

Paramètre	À quoi ça sert	Obligatoire ?
`home:`	Premier écran	Oui (ou `initialRoute`)
`title:`	Nom dans le multitâche	Non
`debugShowCheckedModeBanner:`	Enlever le ruban DEBUG	Non
`theme:`	Couleurs de l'app	Non
`darkTheme:` + `themeMode:`	Mode sombre	Non
`initialRoute:` + `routes:`	Navigation par nom	Non (mais pratique)

L'ordre de la révision

1. `MaterialApp` + `home` + `Scaffold` + `AppBar` + `Text` → tu sais afficher un écran
2. `Navigator.push` + `MaterialPageRoute` → tu sais passer à un autre écran
3. `routes` + `Navigator.pushNamed` → tu sais ranger tes écrans proprement
4. `theme` + `darkTheme` + `themeMode` → tu sais changer les couleurs
5. `onGenerateRoute`, `builder`, `localizationsDelegates` → on revient **plus tard**, quand on en aura besoin

Scaffold

C'est quoi ?

`Scaffold` veut dire **échafaudage** en anglais. C'est **la structure d'UN écran**.

Si `MaterialApp` c'est le **bâtiment entier** (avec ses règles, ses couleurs, sa langue), alors `Scaffold` c'est **une pièce** : le salon, la cuisine, la chambre…

Une app = plusieurs écrans = plusieurs `Scaffold`.

Chaque `Scaffold` te donne **gratuitement** les éléments standards d'un écran mobile :

- Une barre en haut (AppBar)
- Une zone de contenu au milieu (body)

- Une barre en bas (BottomNavigationBar)
- Un bouton flottant (FloatingActionButton)
- Un menu latéral qui glisse (Drawer)
- Un panneau qui glisse depuis la droite (endDrawer)
- Un espace pour le clavier (resizeToAvoidBottomInset)
- etc.

Tu n'as **rien à positionner à la main** : tu remplis les cases, Flutter les place au bon endroit.

Étape 0 — Le strict minimum

```
Scaffold(
  body: Center(child: Text('Bonjour !')),
)
```

Ce que tu as : juste du texte au milieu. Pas de barre, pas de bouton. C'est tout.

Un `Scaffold` peut être **vide** comme ça. Le reste, ce sont des options que tu ajoutes **quand tu en as besoin**.

Important : `Scaffold` doit toujours être **à l'intérieur** d'un `MaterialApp` (sinon erreur "No Material widget found").

Étape 1 — Ajouter une barre en haut (AppBar)

```
Scaffold(
  appBar: AppBar(                      //  ajoute cette ligne
    title: Text('Mon écran'),
  ),
  body: Center(child: Text('Bonjour !')),
)
```

Effet : une barre colorée en haut avec le titre. Le corps passe automatiquement **en dessous**.

Étape 2 — Ajouter un bouton flottant

```
Scaffold(
  appBar: AppBar(title: Text('Mon écran')),
  body: Center(child: Text('Bonjour !')),
  floatingActionButton: FloatingActionButton(   //  ajoute ce bloc
    onPressed: () {
      print('Cliqué !');
```

```
    },
    child: Icon(Icons.add),
  ),
)
```

Effet : un bouton rond en bas à droite, qui flotte au-dessus du contenu. Le classique bouton "+" des apps.

Étape 3 — Ajouter un menu latéral (Drawer)

```
Scaffold(
  appBar: AppBar(title: Text('Mon écran')),
  drawer: Drawer(                              //  ajoute ce bloc
    child: ListView(
      children: [
        DrawerHeader(child: Text('Menu')),
        ListTile(title: Text('Accueil')),
        ListTile(title: Text('Paramètres')),
      ],
    ),
  ),
  body: Center(child: Text('Bonjour !')),
)
```

Effet : une **icône ≡** apparaît automatiquement à gauche de l'AppBar. En cliquant dessus, un panneau glisse depuis la gauche.

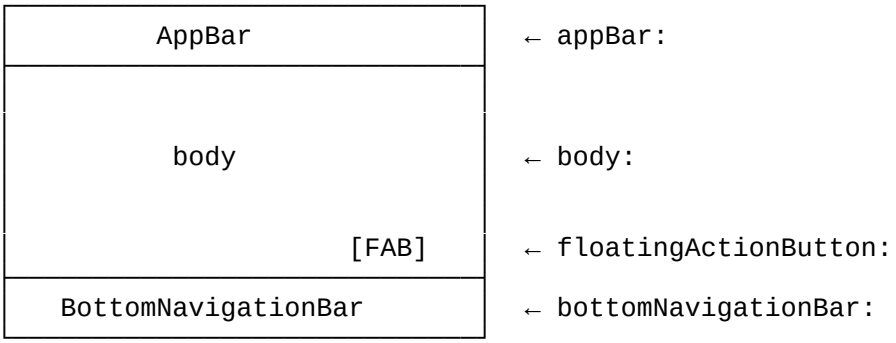
Tu n'as **rien à coder** pour le bouton **≡** → `Scaffold` le crée tout seul dès que tu fournis un `drawer`.

Étape 4 — Ajouter une barre en bas

```
Scaffold(
  appBar: AppBar(title: Text('Mon écran')),
  body: Center(child: Text('Bonjour !')),
  bottomNavigationBar: BottomNavigationBar(    //  ajoute ce bloc
    items: [
      BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Accueil'),
      BottomNavigationBarItem(icon: Icon(Icons.person), label: 'Profil'),
    ],
  ),
)
```

Effet : une barre fixe tout en bas avec des onglets. Le corps se **rétrécit** automatiquement pour laisser la place.

Récap visuel : les zones d'un Scaffold



Tous les paramètres importants

Paramètre	Rôle	Exemple
`appBar`	Barre en haut	`AppBar(title: Text('...'))`
`body`	Contenu principal	N'importe quel widget
`floatingActionButton`	Bouton rond flottant	`FloatingActionButton(...)`
`floatingActionButtonLocation`	Où le placer	`FloatingActionButtonLocation.centerDock`
`drawer`	Menu qui glisse depuis la gauche	`Drawer(child: ...)`
`endDrawer`	Menu qui glisse depuis la droite	`Drawer(child: ...)`
`bottomNavigationBar`	Barre en bas	`BottomNavigationBar(...)`

Paramètre	Rôle	Exemple
`bottomSheet`	Feuille qui reste affichée en bas	`Container(...)`
`backgroundColor`	Couleur de fond	`Colors.grey[200]`
`resizeToAvoidBottomInset`	Le body remonte quand le clavier apparaît	`true` (défaut) / `false`
`extendBody`	Le body passe **derrière** la bottom bar	`true` / `false`
`extendBodyBehindAppBar`	Le body passe **derrière** l'AppBar	`true` / `false`
`persistentFooterButtons`	Boutons fixes juste au-dessus du bas	`[ElevatedButton(...)]`

Les 3 pièges classiques du débutant

Piège 1 — Oublier `MaterialApp` au-dessus

```
//  ERREUR : No Material widget found
runApp(Scaffold(body: Text('Salut')));
```

`Scaffold` a besoin d'un `MaterialApp` **au-dessus** de lui pour fonctionner.

Piège 2 — Le clavier cache le contenu

Par défaut, quand tu cliques dans un `TextField` en bas de l'écran, le clavier cache le champ.

Solution simple : mets ton contenu dans un `SingleChildScrollView` :

```
Scaffold(
  body: SingleChildScrollView(       //  enveloppe
    child: Column(
      children: [
        TextField(),
        // ... beaucoup de champs
      ],
    ),
```

```
  ),
)
```

Le `Scaffold` remonte automatiquement le body quand le clavier apparaît (`resizeToAvoidBottomInset: true` par défaut).

Piège 3 — Confondre `Scaffold` et `MaterialApp`

	`MaterialApp`	`Scaffold`
Combien par app ?	**1 seul**	**1 par écran**
Rôle	Config globale (thème, langue, navigation)	Structure d'un écran
Contient	Des routes, des thèmes	Une AppBar, un body…
Se met où ?	À la racine (`runApp`)	Dans chaque page

Analogie pour bien mémoriser

- `MaterialApp` = **le bâtiment** avec ses règles (couleurs, langue, plan des étages)
- `Scaffold` = **une pièce** du bâtiment
- `AppBar` = **le panneau au-dessus de la porte** de la pièce
- `body` = **le sol** de la pièce
- `drawer` = **une porte secrète** sur le côté
- `bottomNavigationBar` = **le tapis** au fond de la pièce
- `FloatingActionButton` = **un meuble flottant** au-dessus du sol

Exemple complet — tous les paramètres ensemble

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MonApp());
}

class MonApp extends StatelessWidget {
  const MonApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const MonEcran(),
    );
```

```
  }
}

class MonEcran extends StatelessWidget {
  const MonEcran({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(

      //  Barre du haut
      appBar: AppBar(
        title: const Text('Mon écran'),
      ),

      //  Menu latéral gauche (icône ≡ automatique)
      drawer: Drawer(
        child: ListView(
          children: const [
            DrawerHeader(child: Center(child: Text('Menu'))),
            ListTile(leading: Icon(Icons.home), title: Text('Accueil')),
            ListTile(leading: Icon(Icons.settings), title: Text('Réglages')),
          ],
        ),
      ),

      //  Contenu principal
      body: const Center(
        child: Text('Bonjour !', style: TextStyle(fontSize: 24)),
      ),

      //  Bouton rond flottant
      floatingActionButton: FloatingActionButton(
        onPressed: () => print('Ajout !'),
        child: const Icon(Icons.add),
      ),

      // Barre en bas
      bottomNavigationBar: BottomNavigationBar(
        items: const [
          BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Accueil'),
          BottomNavigationBarItem(icon: Icon(Icons.person), label: 'Profil'),
        ],
      ),
    );
  }
}
```

Ce qu'on doit retenir (version courte)

1. `Scaffold` = **structure d'un écran**
2. Il a **5 zones principales** : appBar, body, drawer, FAB, bottomNavigationBar
3. Tu **remplis seulement les zones dont tu as besoin** → les autres disparaissent

4. Il **doit** être dans un `MaterialApp`
5. Le clavier : entoure ton body de `SingleChildScrollView` si tu as des champs en bas

AppBar

C'est quoi ?

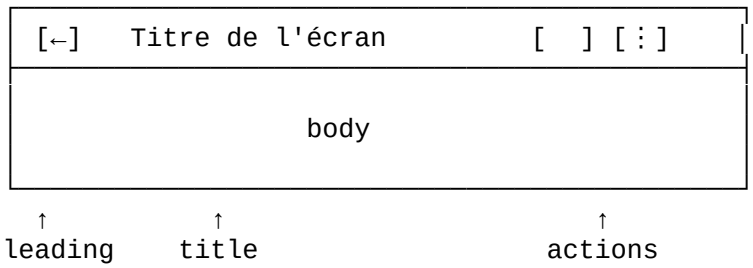
`AppBar` = **la barre en haut d'un écran**. Celle où il y a le titre, la flèche retour, parfois des icônes à droite.

Tu la mets dans le paramètre `appBar:` d'un `Scaffold` :

```
Scaffold(
  appBar: AppBar(title: Text('Mon écran')),
  body: ...,
)
```

`AppBar` ne marche **pas** tout seul. Il a besoin d'être **dans un Scaffold** (sinon erreur).

À quoi ça ressemble (schéma)



Trois zones principales :

- **leading** (gauche) → flèche retour, ou icône ≡
- **title** (centre) → le texte principal
- **actions** (droite) → liste d'icônes/boutons

Étape 0 — Le minimum

```
AppBar(title: Text('Mon écran'))
```

Effet : une barre colorée en haut avec un texte centré (ou à gauche selon la plateforme).

Étape 1 — Ajouter des icônes à droite (`actions`)

```
AppBar(
  title: Text('Mon écran'),
  actions: [                          //  liste d'icônes à droite
    IconButton(
      icon: Icon(Icons.search),
      onPressed: () => print('Recherche'),
    ),
    IconButton(
      icon: Icon(Icons.more_vert),
      onPressed: () => print('Menu'),
    ),
  ],
)
```

Effet : deux icônes apparaissent à droite du titre. **L'ordre** de la liste = l'ordre de gauche à droite.

Étape 2 — Remplacer le bouton retour (`leading`)

Par défaut, quand tu navigues vers un nouvel écran, Flutter met **automatiquement** une flèche ← à gauche.

Pour la **remplacer** par autre chose (ex : un ≡ pour ouvrir un drawer) :

```
AppBar(
  leading: IconButton(                //  remplace la flèche par défaut
    icon: Icon(Icons.menu),
    onPressed: () => print('Menu'),
  ),
  title: Text('Mon écran'),
)
```

Attention : si tu définis `leading` **ET** un `drawer` dans le Scaffold, c'est ton `leading` qui gagne. Le ≡ auto **n'apparaît plus**.

Étape 3 — Mettre une barre de recherche / onglets en dessous (`bottom`)

```
AppBar(
  title: Text('Mon écran'),
  bottom: TabBar(                    //  un widget SOUS le titre
    tabs: [
      Tab(text: 'Accueil'),
      Tab(text: 'Profil'),
    ],
  ),
)
```

Effet : l'AppBar devient **plus haute** et affiche des onglets juste en dessous du titre.

Tu peux mettre n'importe quel widget dans `bottom:` (un `TextField` pour une barre de recherche, un `PreferredSize` custom, etc.).

Étape 4 — Changer la couleur

```
AppBar(
  title: Text('Mon écran'),
  backgroundColor: Colors.green,      //  couleur de fond
  foregroundColor: Colors.white,      //  couleur du texte et des icônes
)
```

Effet : AppBar verte avec texte blanc.

`foregroundColor` change **tout d'un coup** : titre, icônes, flèche retour. Beaucoup plus simple que de colorier chaque élément.

Étape 5 — Enlever l'ombre (`elevation`)

```
AppBar(
  title: Text('Mon écran'),
  elevation: 0,                       //  pas d'ombre sous la barre
)
```

Effet : la barre se fond dans le body, sans ligne de séparation. Très courant dans le design moderne.

Par défaut, `elevation` est à 4 (une légère ombre apparaît quand tu scrolles le contenu).

Étape 6 — Centrer ou non le titre

```
AppBar(
  title: Text('Mon écran'),
  centerTitle: true,                  //  titre centré (défaut iOS)
)
```

- `true` → titre au centre (style iPhone)
- `false` → titre à gauche (style Android, par défaut)

Tous les paramètres importants

Paramètre	Rôle	Exemple
`title`	Texte/Widget principal	`Text('Accueil')`
`leading`	Widget à gauche (remplace la flèche)	`IconButton(...)`
`actions`	Liste de widgets à droite	`[IconButton(...), IconButton(...)]`
`bottom`	Widget SOUS le titre	`TabBar(...)`, `PreferredSize(...)`
`backgroundColor`	Couleur de fond	`Colors.green`
`foregroundColor`	Couleur texte + icônes	`Colors.white`
`elevation`	Ombre sous la barre	`0`, `4`, `8`
`centerTitle`	Titre centré ?	`true` / `false`
`automaticallyImplyLeading`	Mettre la flèche retour auto ?	`true` (défaut) / `false`
`toolbarHeight`	Hauteur de la barre	`56` (défaut), `80`
`shape`	Forme custom (bord arrondi, etc.)	`RoundedRectangleBorder(...)`
`flexibleSpace`	Couche d'arrière-plan (gradient, image)	`FlexibleSpaceBar(...)`

Les pièges classiques du débutant

Piège 1 — `AppBar` sans `Scaffold` → erreur

```
// ERREUR : No Material widget found
runApp(AppBar(title: Text('Salut')));
```

`AppBar` **doit** être dans le paramètre `appBar:` d'un `Scaffold`.

Piège 2 — Vouloir cacher la flèche retour

Par défaut, quand tu fais `Navigator.push(...)` vers un nouvel écran, la flèche ← apparaît **toute seule**. Pour la supprimer :

```
AppBar(
  title: Text('Mon écran'),
  automaticallyImplyLeading: false,    //  enlève la flèche auto
)
```

Piège 3 — Mettre un `drawer` ET un `leading` en même temps

```
//  Le ≡ automatique n'apparaîtra PAS si tu définis leading
Scaffold(
  appBar: AppBar(
    leading: IconButton(icon: Icon(Icons.search), onPressed: ...),
  ),
  drawer: Drawer(...),   // le ≡ auto est "écrasé" par leading
)
```

Si tu veux **garder** le ≡ automatique du drawer, **ne touche pas** à `leading`.

Piège 4 — Titre trop long qui déborde

Un `Text` long dans `title:` peut provoquer un débordement visuel. Solution :

```
AppBar(
  title: Text(
    'Un titre vraiment très très long',
    overflow: TextOverflow.ellipsis,   //  coupe avec "..."
  ),
)
```

Analogie pour mémoriser

`AppBar` = **le panneau d'entrée d'une pièce** :

- `leading` → la **poignée de porte** à gauche
- `title` → l'**étiquette** au milieu
- `actions` → les **boutons/interrupteurs** à droite
- `bottom` → une **affiche collée sous le panneau**
- `backgroundColor` / `foregroundColor` → la **peinture** du panneau

Exemple complet — tous les paramètres ensemble

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MonApp());
}

class MonApp extends StatelessWidget {
  const MonApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(primarySwatch: Colors.deepPurple),
      home: const MonEcran(),
    );
  }
}

class MonEcran extends StatelessWidget {
  const MonEcran({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      //  AppBar avec plein d'options
      appBar: AppBar(
        //  Titre
        title: const Text('Mon AppBar'),

        //  Titre centré
        centerTitle: true,

        //  Icône à gauche
        leading: IconButton(
          icon: const Icon(Icons.arrow_back),
          onPressed: () => print('Retour'),
        ),
```

```
        //  Icônes à droite
        actions: [
          IconButton(
            icon: const Icon(Icons.search),
            onPressed: () => print('Recherche'),
          ),
          IconButton(
            icon: const Icon(Icons.more_vert),
            onPressed: () => print('Menu'),
          ),
        ],

        //  Couleurs
        backgroundColor: Colors.deepPurple,
        foregroundColor: Colors.white,

        //  Pas d'ombre
        elevation: 0,

        //  Onglets en dessous du titre
        bottom: const TabBar(
          indicatorColor: Colors.white,
          labelColor: Colors.white,
          unselectedLabelColor: Colors.white70,
          tabs: [
            Tab(text: 'Accueil'),
            Tab(text: 'Profil'),
            Tab(text: 'Réglages'),
          ],
        ),
      ),

      body: const Center(child: Text('Contenu de l\'écran')),
    );
  }
}
```

Resultat : AppBar violette avec :

- une flèche retour à gauche
- un titre centré
- deux icônes à droite
- trois onglets en dessous

Ce qu'on doit retenir (version courte)

1. `AppBar` = **barre du haut d'un écran**
2. **3 zones principales** : `leading` (gauche), `title` (centre), `actions` (droite)
3. `bottom:` = widget **sous** le titre (souvent un `TabBar`)
4. `foregroundColor` = **tout colorier d'un coup** (texte + icônes)

5. La flèche retour apparaît **toute seule** → mets `automaticallyImplyLeading: false` pour la virer
6. Elle doit être **dans un Scaffold**